BSEP 2021/2022

VEŽBE 6

OWASP TOP TEN

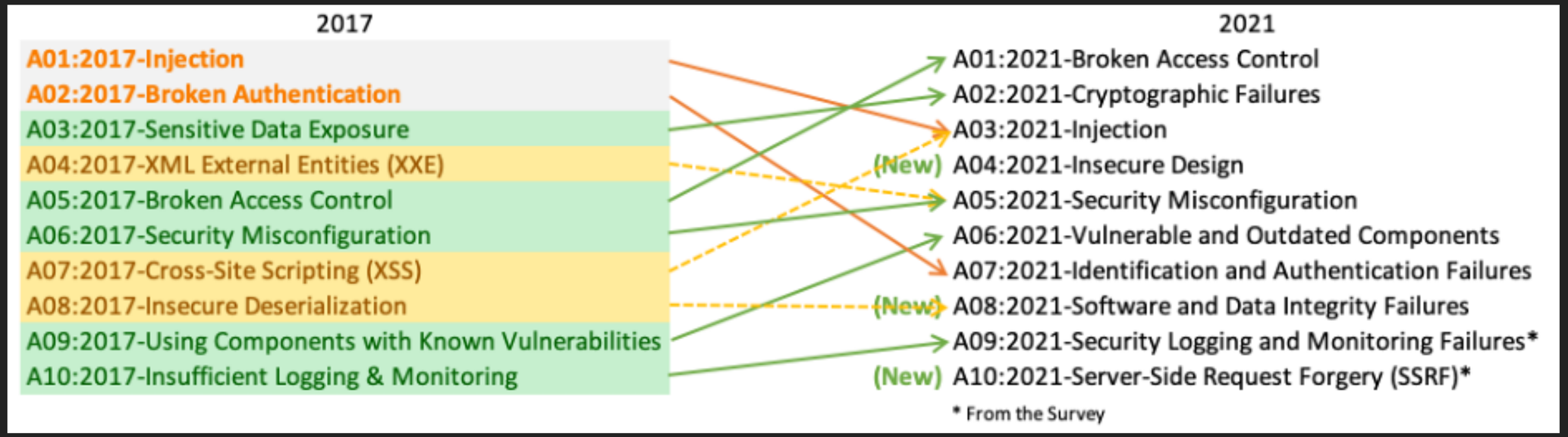
OWASP

OWASP - The Open Web Application Security Project

Neprofitna organizacija čija je misija da pruži pomoć i obuku programerima i kompanijama u cilju unapređenja bezbednosti

OWASP TOP TEN

OWASP Top Ten je dokument koji sadrži listu najrasprostranjenijih i najozbiljnijih bezbednosnih problema u današnjim veb-baziranim sistemima



OWASP Top 10

2017

OWASP Top 10

2021

OWASP TOP 10

01 Broken Access Control

02 Cryptographic Failures

03 Injection

04 Insecure Design

05 Security Misconfiguration

06 Vulnerable and Outdated Components

07 Identification and Authentication Failures

08 Software and Data Integrity Failures

09 Security Logging and Monitoring Failures

10 Server Side Request Forgery (SSRF)

01 Broken Access Control

Korisnik je slučajno dobio pristup nekoj funkciji, a nije smeo

Napadač je pristupio stranici koja je namenjena za administratora: https://example.com/app/admin_getappInfo

Deny access by default, except for public resources

Build strong access control mechanisms and reuse them across the application

Disable server directory listing and do not store sensitive data in root

Rate limit API and controller access

Validate JWT tokens after logout

02 Cryptographic Failures

Sensitive Data Exposure (2017)

Problem nastaje ako nije implementirana zaštita podataka u tranzitu i skladištu

Identify sensitive data and apply appropriate security controls

Don’t store sensitive data unless absolutely needed—discard sensitive data, use tokenization or truncation

Encrypt all sensitive data at rest using strong encryption algorithms, protocols and keys

Encrypt data in transit using secure protocols like TLS and HTTP HSTS

Disable caching for sensitive data

Store passwords using strong, salted hashing functions like Argon2, scrypt and bcrypt

03 Injection

Napadači ubrizgavaju kod u podatke koji se šalju interpreteru

Use a safe API which avoids the use of the interpreter entirely

Use positive or “whitelist” server-side input validation

Escape special characters

Use LIMIT and other SQL controls within queries to prevent mass disclosure of records in case of SQL injection.

04 Insecure Design

Ranjivost se desi jer nedostaje ili je neefektivna određena bezbednosna kontrola

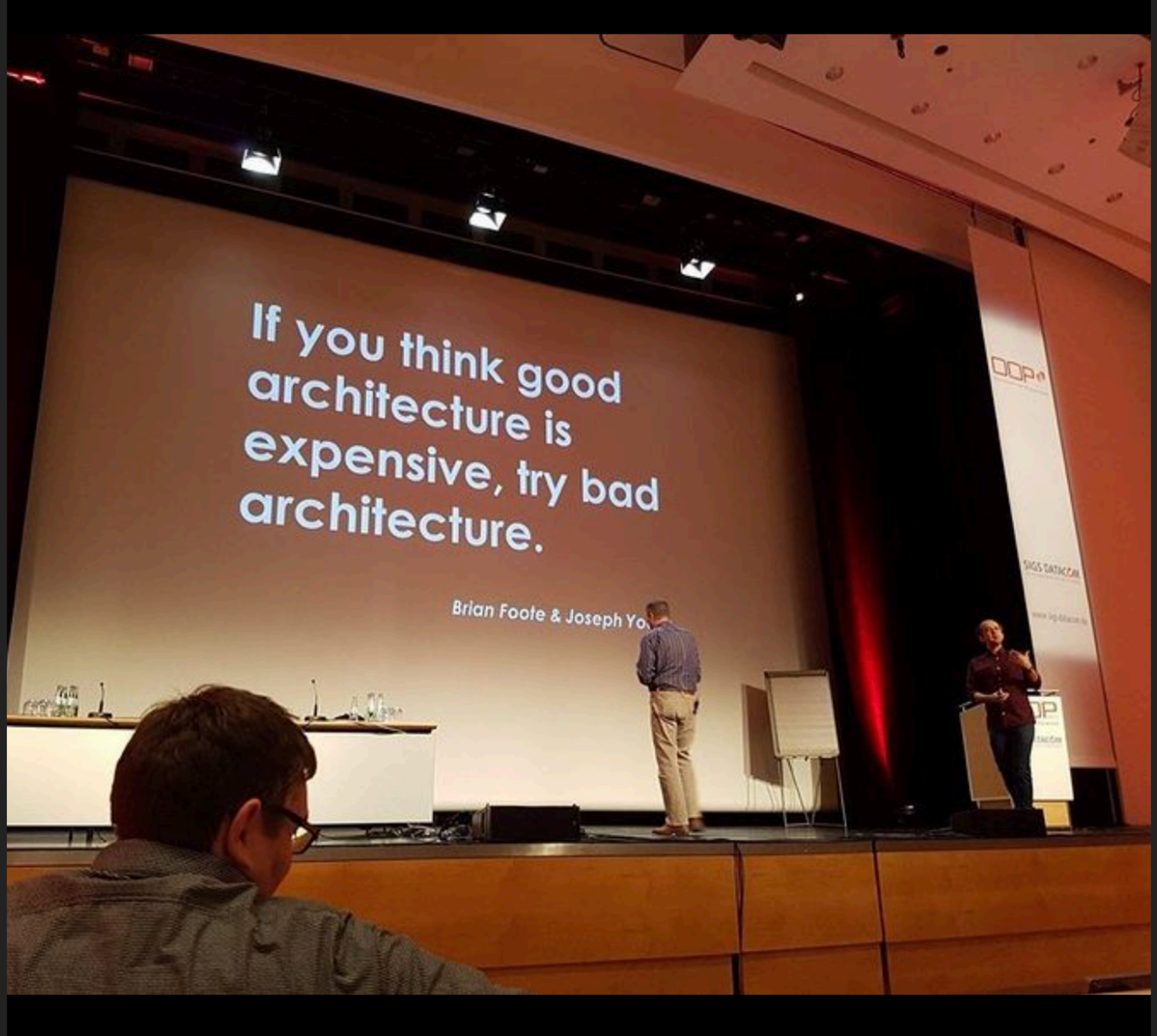
Establish a secure software development lifecycle (SSDLC)

Leverage application security practices from early stages of software development

Create a library of secure design patterns, and use it to build new applications

Leverage threat modeling to design critical features like authentication and access control

Integrate security concerns and controls into all user stories


If you think good architecture is expensive, try bad architecture.
Brian Foote & Joseph Yo

Loša konfiguracija

Establish a hardening process for applications, which is fast and easy to deploy

Configure development, QA, and production identically (with different credentials)

All systems should have a minimal setup without unnecessary features and components

Configurations should be regularly updated, applying patches and security advisories

Establish an automated process to verify secure configurations in all environments

06 Vulnerable and Outdated Components

Using Components with Known Vulnerabilities (2017)

Upotreba komponenti, biblioteka koje imaju ranjivosti i zastarele su

Remove unused dependencies, features, components, and files from applications.

Maintain an inventory of components and their versions, both on the client side and server side, using software composition analysis (SCA) tools

Continuously scan libraries and their dependencies for vulnerable components

Only use components from official sources, and prefer signed packages

Urgently remediate vulnerabilities, remove affected components, or apply a virtual patch

07 Identification and Authentication Failures

Broken Authentication (2017)

Autentifikacija nije dobro implementirana

Implement multi-factor authentication

Do not deploy systems with default credentials

Check for a list of the top 10,000 worst passwords

Use the guidelines in NIST 800-63 B section 5.1.1 for Memorized Secrets

Harden all authentication-related processes like registration and credential recovery

Limit or delay failed login attempts

TIME IT TAKES A HACKER TO BRUTE FORCE YOUR PASSWORD IN 2022

Number of Characters	Numbers Only	Lowercase Letters	Upper and Lowercase Letters	Numbers, Upper and Lowercase Letters	Numbers, Upper and Lowercase Letters, Symbols
4	Instantly	Instantly	Instantly	Instantly	Instantly
5	Instantly	Instantly	Instantly	Instantly	Instantly
6	Instantly	Instantly	Instantly	Instantly	Instantly
7	Instantly	Instantly	2 secs	7 secs	31 secs
8	Instantly	Instantly	2 mins	7 mins	39 mins
9	Instantly	10 secs	1 hour	7 hours	2 days
10	Instantly	4 mins	3 days	3 weeks	5 months
11	Instantly	2 hours	5 months	3 years	34 years
12	2 secs	2 days	24 years	200 years	3k years
13	19 secs	2 months	1k years	12k years	202k years
14	3 mins	4 years	64k years	750k years	16m years
15	32 mins	100 years	3m years	46m years	1bn years
16	5 hours	3k years	173m years	3bn years	92bn years
17	2 days	69k years	9bn years	179bn years	7tn years
18	3 weeks	2m years	467bn years	11tn years	438tn years

08 Software and Data Integrity Failures

Kod i infrastruktura koji su podložni kršenju integriteta

Use digital signatures or similar mechanisms to verify software or data is from the expected source and has not been altered

Ensure libraries and dependencies, such as npm or maven, are pulling from trusted repositories

Establish a review process for code and configuration changes

Ensure that your CI/CD pipeline has proper configuration and access controls

09 Security Logging and Monitoring Failures

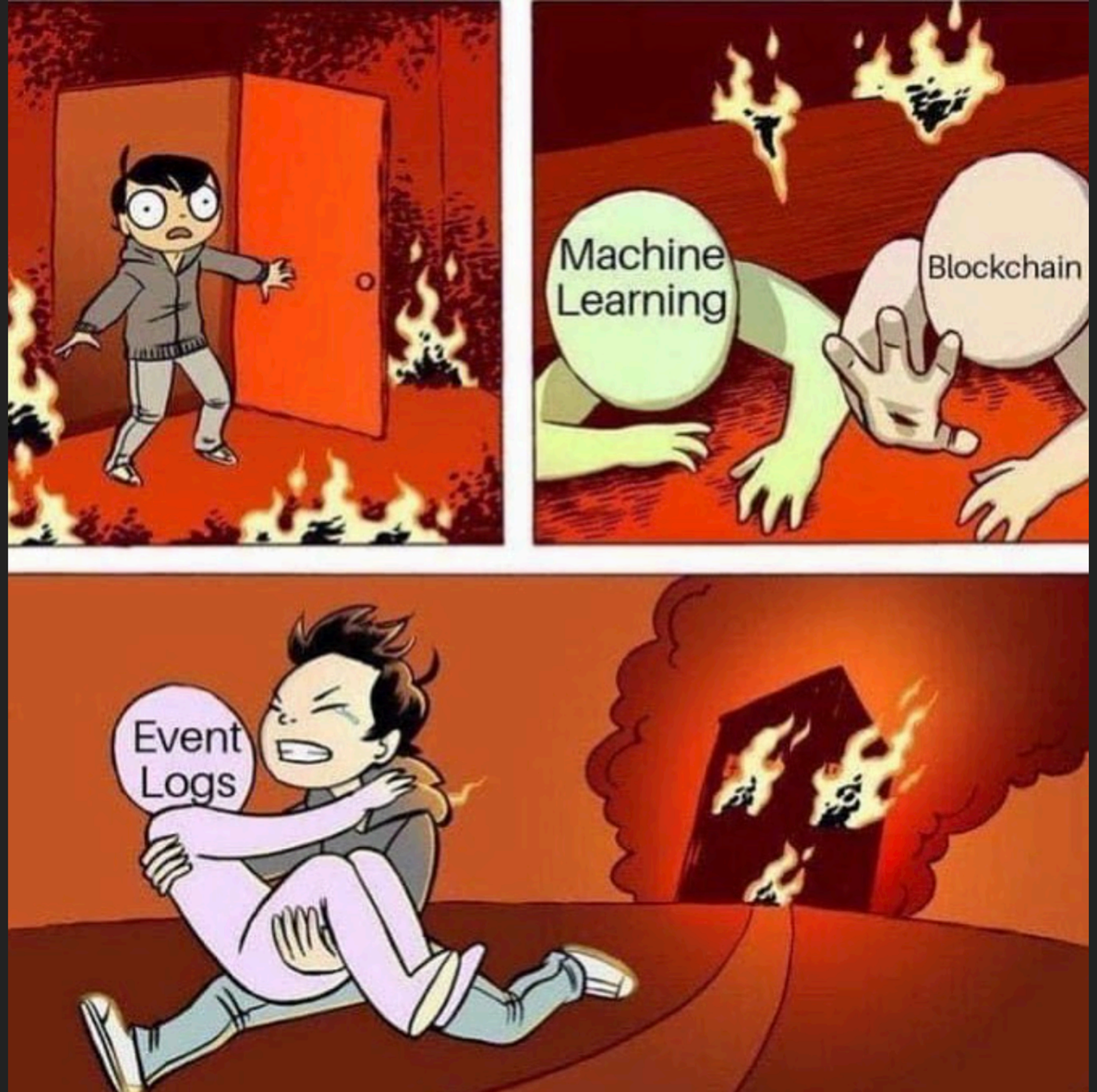
Implementirati loging i monitoring

Ensure login, access control, and server-side input validation is logged

Ensure logs contain enough context to identify suspicious behavior and enable in-depth forensic analysis

Ensure logs are in a format compatible with log management solutions

Take measures to prevent attackers from tampering with log data


Machine Learning
Blockchain
Event Logs

10 Server Side Request Forgery (SSRF)

Problem nastaje kada aplikacija povuče podatke bez prethodne validacije URL-a

Avoid accepting URLs in client inputs, and if absolutely necessary, sanitize inputs

Isolate any remote resource access functionality in a separate network to reduce impact

Use “deny by default” firewall policies to block unwanted Internet traffic

Use a positive allow list with URL schema, port, and destination

Disable HTTP redirections

Never return raw responses to clients